

## 0.47.3. Carre — les réglages d'un carré sous un nom

`#include <Carre>` : `Carre ronde = { x, y, tuile, taille, sens, vitesse, tours };` puis `carre(ronde)` ou `losange(ronde)`.

### 1. Ce qu'il faut comprendre

**Ce que c'est** : un **Carre** range les **sept réglages** d'un carré sous un seul nom. `carre(ronde);` vaut `carre(10, 8, ALPHABET[0], 2, 1, 250, 1);` — et `losange(ronde);` marche aussi.

**La ligne à écrire** : `#include <Carre>`, avec un C majuscule, comme on l'écrit. Il faut aussi `<carre>` pour s'en servir.

**Ce qu'il coûte** : rien de plus que `carre()` : le compilateur recopie les sept réglages à la place du nom.

**Essaie** : change seulement le sens dans `{ ... }` : -1.

**On retrouve Carre dans 6 leçons**

[0.48](#), [0.48.1](#), [0.48.2](#), [0.54](#), [0.54.1](#), [0.54.2](#)

### 2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1349 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage – la mise en route de la console
carre()         440 o ← main() – une fonction fournie par le langage
main()          38 o ← démarrage – ta fonction
VBlank          8 o ← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage   28 o ← image() – attend l'image suivante
AttendreVBlank  16 o ← démarrage – attend que l'écran soit libre
AttendreAcces   12 o ← EffacerCases – attend de pouvoir écrire une case
AdresseCase     10 o ← poser(), effacer() – trouve l'adresse d'une case de l'écran
EffacerCases    12 o ← effacer() – efface des cases
CopierTuiles    18 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte    14 o ← démarrage – vide les deux cartes de l'écran
Tuiles          704 o ← CopierTuiles – la police entière
```

Lettres gravées : toutes (poser() reçoit un numéro calculé)

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```
poser() × 4      52 o sur place, dans carre() + AdresseCase (10 o)
effacer() × 3    45 o sur place, dans carre() + AdresseCase (10 o), EffacerCases (12 o)
image() × 4      12 o sur place, dans carre(), main() + AttendreImage (28 o)
```

Tes fonctions appelées :

```
carre() × 1      31 o sur place, dans main()
```

## 1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- #include <Carre> : les réglages d'un carré sous un nom ----  
  
#include <Carre>      // Carre : sept réglages sous un seul nom  
#include <carre>  
#include <ALPHABET>
```

## 2. UNE VARIABLE — RONDE

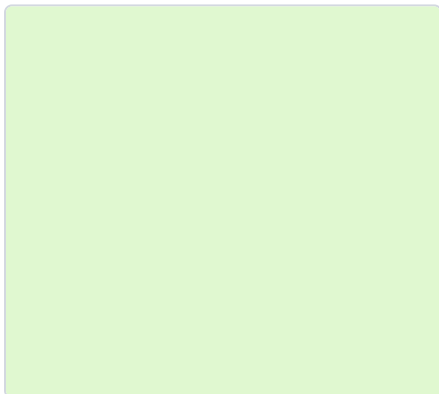
```
Carre ronde = { 10, 8, ALPHABET[0], 2, 1, 150, 1 }; // centre, tuile, taille 2, sens, vitesse, tours
```

## 3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

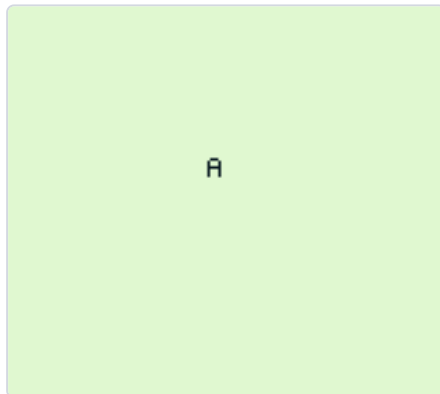
```
int main() {  
    carre(ronde); // = carre(10, 8, ALPHABET[0], 2, 1, 150, 1);  
  
    while (true) {  
        image();  
    }  
}
```

## 3. Ce qui se passe, étape par étape

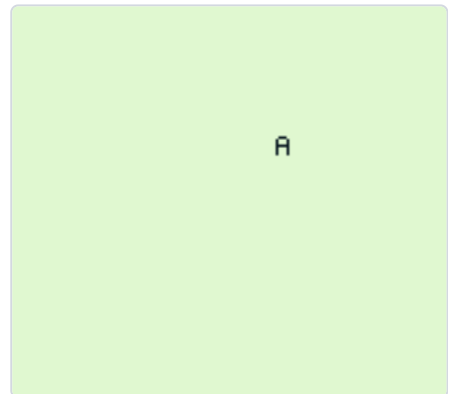
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



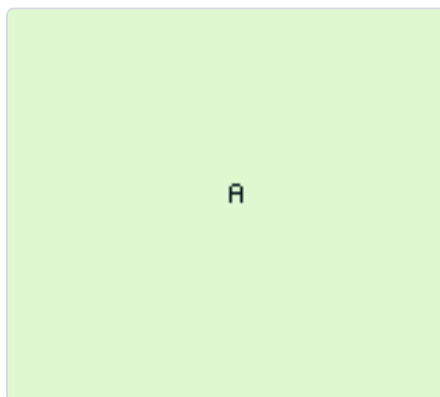
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

## 4. Ce qu'on doit voir

---

Un A qui fait un grand tour en carré ( $5 \times 5$ ) autour du milieu.

## 5. Ce que la console répond

---

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le A passe par les coins du carré de taille 2

✓ il revient au centre

---

Ensuite : **0.48. Les réglages sous un nom : Carre** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++